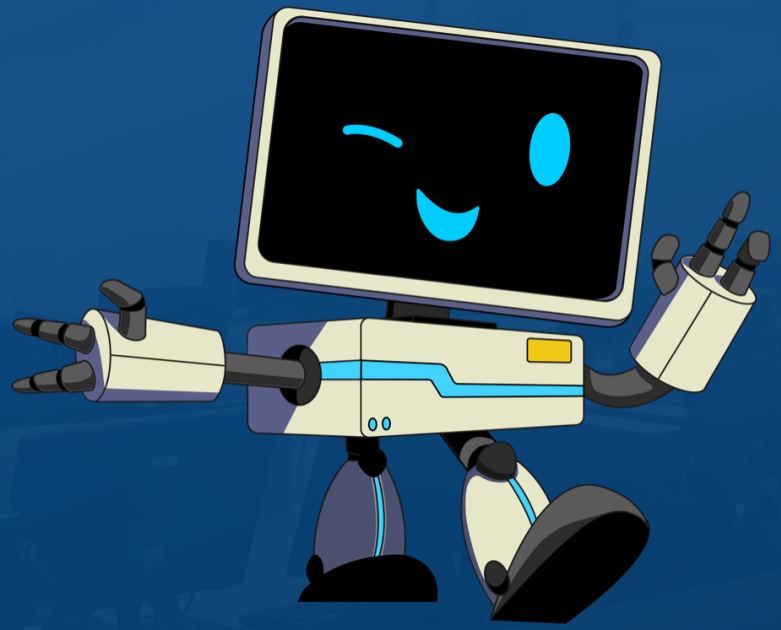
UIFCE
UNIDAD DE INFORMÁTICA



**Curso:**
Introducción a Python y R

**Sesión 3:**
Listas, bucles y funciones

INTRODUCCIÓN A PYTHON Y R · SESIÓN 3

# Repetir sin repetirse


**Nombre del monitor(a): Yony Sebastián Chaparro Mesa**

Estudiante Auxiliar

***Correo:*** uniic_bog@unal.edu.co

***Tel:*** 3165000 ***Ext:*** 12301



*Facultad de Ciencias Económicas*
*Unidad de Informática*
*Sede Bogotá*

UNIVERSIDAD NACIONAL DE COLOMBIA

# Contenido

Sesión 3 · Python en Google Colab

1. **Repaso activo**
2. **Listas**
3. **El bucle for**
4. **Acumuladores y range()**
5. **Funciones**
6. **¡A practicar!**

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301



UIFCE
UNIDAD DE INFORMÁTICA

1

# Repaso activo

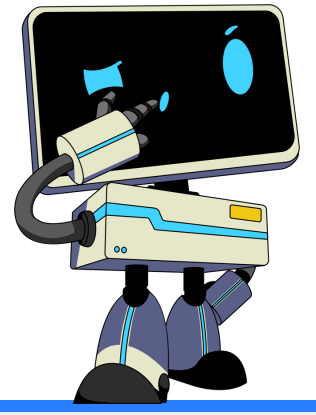
Lo que vimos en la sesión 2

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

# Calentamiento

**RETO**

Escribe un programa que clasifique una nota: **“Excelente”** si es 4.5 o más, **“Aprobó”** si es 3.0 o más y **“Reprobó”** en otro caso.



**Con `nota = 3.8`, ¿qué imprime tu programa?**

```
nota = 3.8
if nota >= 4.5:
    print("Excelente")
elif nota >= 3.0:
    print("Aprobó")
else:
    print("Reprobó")
```

Imprime **“Aprobó”**: 3.8 no llega a 4.5, pero sí es mayor o igual a 3.0.

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301



UIFCE
UNIDAD DE INFORMÁTICA

# 2

# Listas

Muchos datos en una sola variable

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

# ¿Por qué necesitamos listas?

## Una variable por dato

Funciona con 4 notas... ¿y con 40?

```
nota1 = 3.5
nota2 = 4.2
nota3 = 2.8
nota4 = 4.9
```

## Una lista

Todos los datos juntos, en orden.

```
notas = [3.5, 4.2, 2.8, 4.9]
```

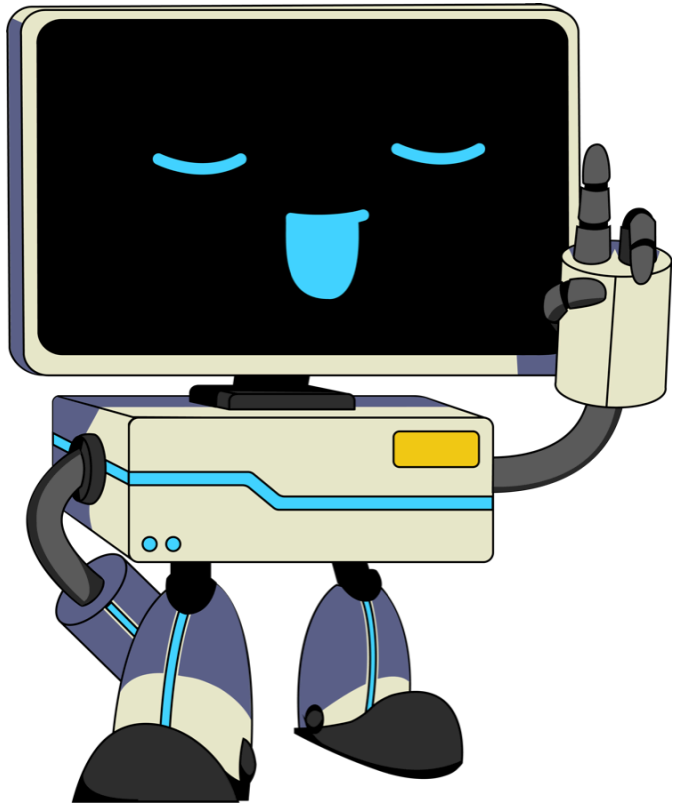
Una **lista** guarda **varios valores en orden** dentro de una sola variable.

# Crear una lista

Corchetes `[ ]` y elementos separados por comas

```
notas = [3.5, 4.2, 2.8, 4.9]
ciudades = ["Bogotá", "Medellín", "Cali"]
vacia = []

print(ciudades)
# ['Bogotá', 'Medellín', 'Cali']
```



- Puede guardar números, textos o cualquier otro tipo de dato.
- Una lista también puede empezar **vacía** e ir creciendo.

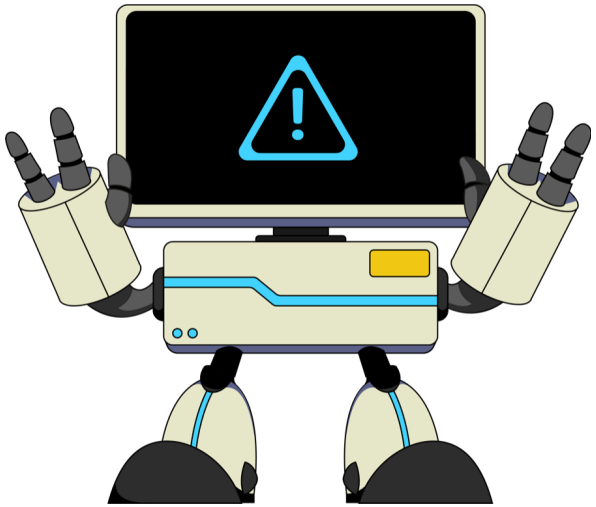
# Las posiciones empiezan en 0

Cada elemento tiene una posición (índice)

| 3.5 | 4.2 | 2.8 | 4.9 |
|---|---|---|---|
| 0 primero | 1 | 2 | 3 último |

```
print(notas[0])   # 3.5
print(notas[3])   # 4.9
```

**En Python se cuenta desde 0. En R, desde 1: lo veremos en la sesión 5.**



**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301



UIFCE
UNIDAD DE INFORMÁTICA

# Contar y agregar

## `len()` cuenta

Cuántos elementos tiene la lista.

```
notas = [3.5, 4.2, 2.8, 4.9]
print(len(notas))   # 4
```

## `.append()` agrega

Pone un elemento nuevo al final.

```
notas.append(3.0)
print(notas)
# [3.5, 4.2, 2.8, 4.9, 3.0]
print(len(notas))   # 5
```

`append` se escribe **después** de la lista, con un punto: es una acción que la lista sabe hacer sobre sí misma.

# 3

# El bucle for

Repetir una acción para cada elemento

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

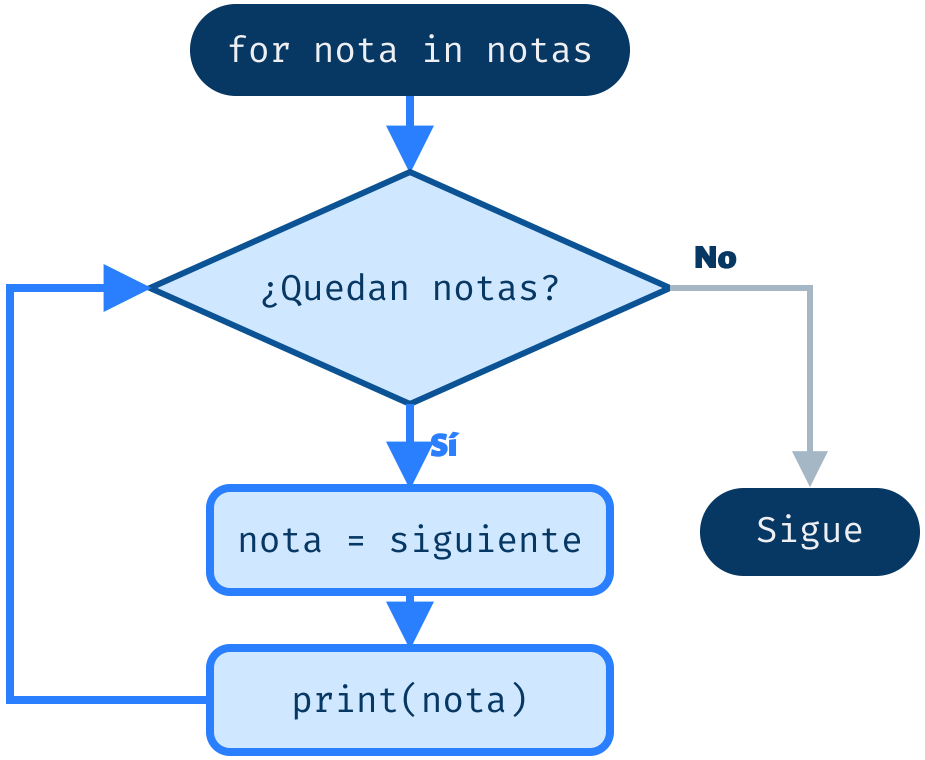
# Anatomía de un for

```
for 1 nota 2 in notas 3: 4
    print("Nota:", nota)
```

```
# Salida
Nota: 3.5
Nota: 4.2
Nota: 2.8
Nota: 4.9
```

1. La palabra **for** (“para cada”).
2. Una **variable** que toma cada valor, uno a la vez. El nombre lo eliges tú.
3. La **lista** que se recorre, después de **in**.
4. Los **dos puntos** y, debajo, el bloque **indentado** que se repite.

# ¿Cómo se ejecuta un for?




| VUELTA | nota | IMPRIME |
|---|---|---|
| 1 | 3.5 | Nota: 3.5 |
| 2 | 4.2 | Nota: 4.2 |
| 3 | 2.8 | Nota: 2.8 |
| 4 | 4.9 | Nota: 4.9 |

# 4

# Acumuladores y range()

Sumar, contar y repetir

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

# El acumulador: sumar en cada vuelta

```
suma = 0                    # empieza en 0
for nota in notas:
    suma = suma + nota      # acumula

promedio = suma / len(notas)
print("Promedio:", promedio)
# Promedio: 3.85
```

El acumulador debe tener un valor **antes** del for.

| VUELTA | nota | suma DESPUÉS |
|---|---|---|
| — | — | 0 |
| 1 | 3.5 | 0 + 3.5 = 3.5 |
| 2 | 4.2 | 3.5 + 4.2 = 7.7 |
| 3 | 2.8 | 7.7 + 2.8 = 10.5 |
| 4 | 4.9 | 10.5 + 4.9 = 15.4 |

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301



UIFCE
UNIDAD DE INFORMÁTICA

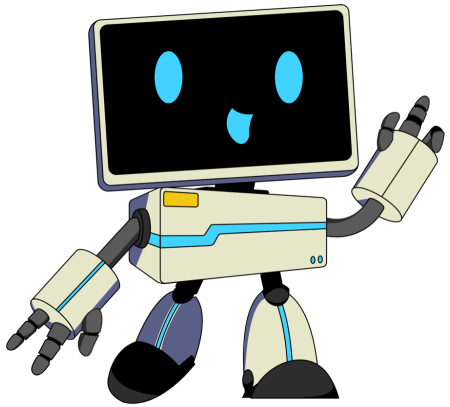
# Bucle + condicional: contar

¿Cuántos estudiantes aprobaron?

```
aprobados = 0
for nota in notas:
    if nota >= 3.0:
        aprobados = aprobados + 1

print("Aprobaron:", aprobados)
# Aprobaron: 3
```

- El `if` va **dentro** del for: se revisa en cada vuelta.
- El contador suma **1**, no la nota.
- Dos niveles de indentación: 4 y 8 espacios.



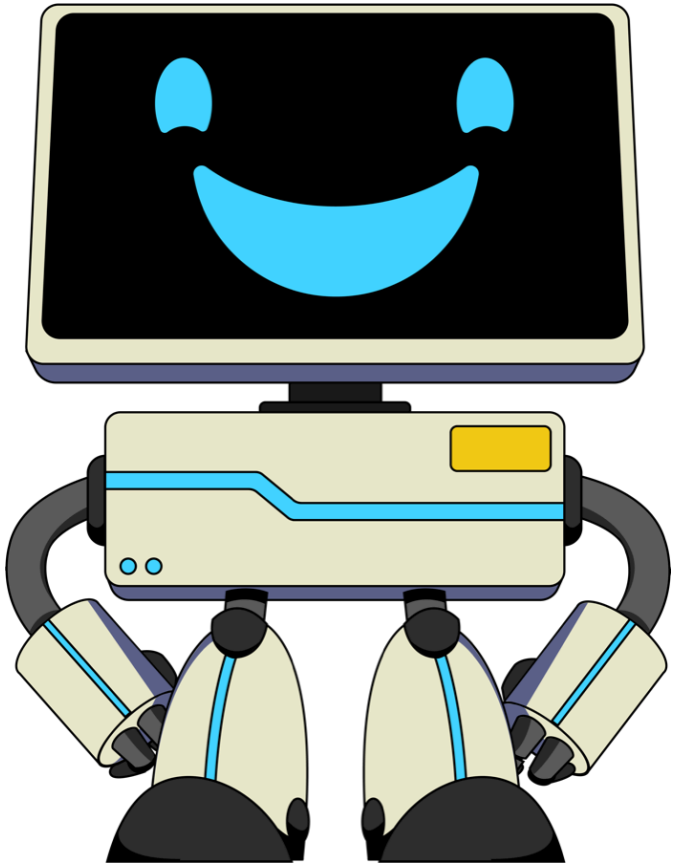
# Práctica en parejas

## El promedio de mis notas

**RETO**

Completa el programa para calcular el **promedio** de la lista. Luego agrega una nota más con `append()` y vuelve a ejecutarlo.

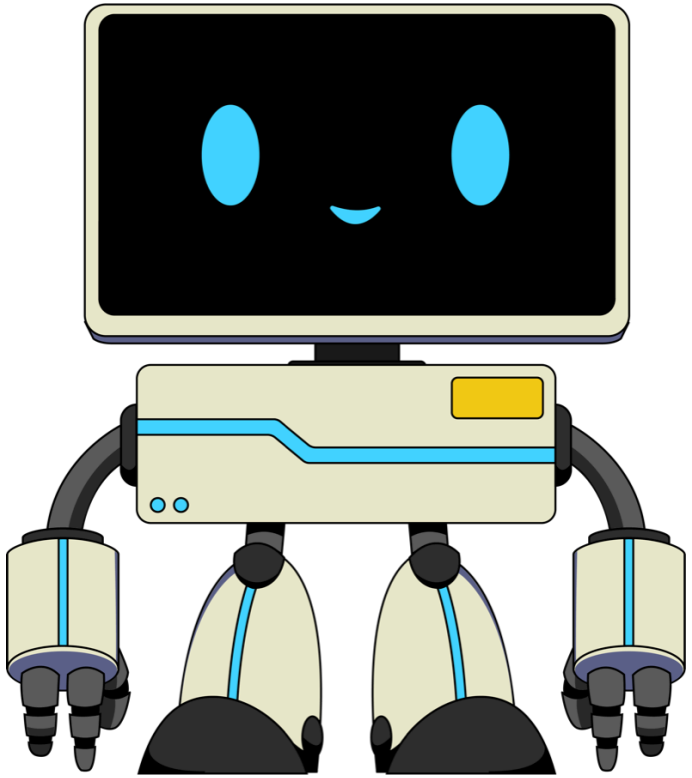
```
notas = [4.0, 3.5, 4.5, 3.0]
suma = ______
for nota in notas:
    suma = ______
print("Promedio:", suma / ______)
```



# Pausa

Estírate y toma agua.

# 10:00



# range(): repetir n veces

```python
for i in range(5):
    print("Repetición", i)
```

```
# Salida
Repetición 0
Repetición 1
Repetición 2
Repetición 3
Repetición 4
```

| CÓDIGO | NÚMEROS QUE GENERA |
|---|---|
| `range(5)` | `0, 1, 2, 3, 4` |
| `range(1, 6)` | `1, 2, 3, 4, 5` |

`range(5)` da **5 números**, pero empieza en 0 y **no incluye el 5**.

# 5

# Funciones

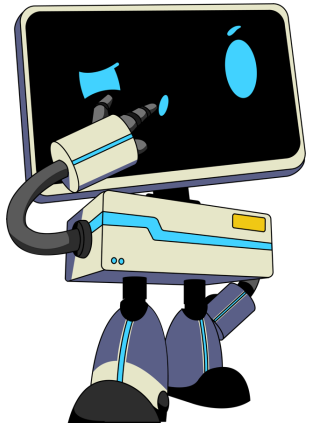
Una receta con nombre

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

# ¿Por qué crear funciones?

**Si calculas el promedio de tres listas, ¿copias el mismo código tres veces?**



### Se escribe una vez

La solución queda guardada con un nombre.

### Se usa muchas veces

Con datos distintos cada vez.

### Se corrige en un lugar

Si hay un error, se arregla una sola vez.

# Anatomía de una función

```
def 1 calcular_promedio 2(lista) 3:
    suma = 0
    for valor in lista:
        suma = suma + valor
    return 4 suma / len(lista)

print(calcular_promedio([3.5, 4.0, 4.5])) 5
# 4.0
```

1. **def** (“definir”) avisa que vamos a crear una función.
2. El **nombre**: con las mismas reglas de las variables.
3. El **parámetro**: el dato que recibe.
4. **return** devuelve el resultado.
5. La **llamada**: aquí se ejecuta de verdad.

# `print` muestra, `return` devuelve

## `print` muestra

El resultado aparece en pantalla, pero no se puede guardar.

```
def saludar(nombre):
    print("Hola,", nombre)

r = saludar("Ana")    # Hola, Ana
print(r)               # None
```

## `return` devuelve

El resultado se puede guardar y usar en otros cálculos.

```
def aplicar_iva(precio):
    return precio * 1.19

total = aplicar_iva(10000)
print(total)          # 11900.0
```

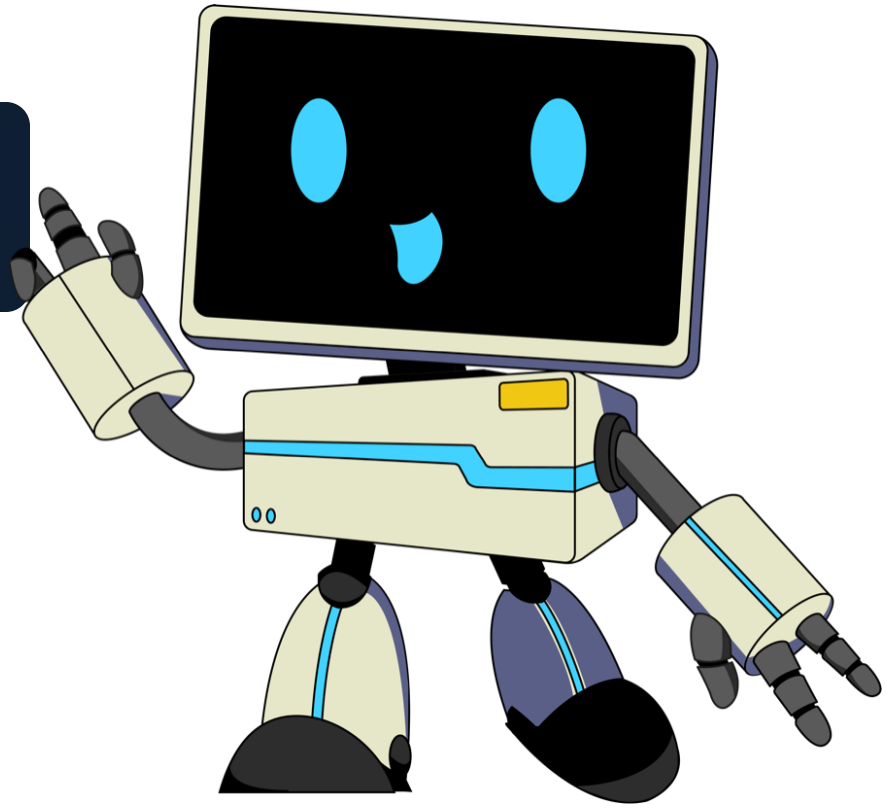
Si la función no tiene `return`, devuelve `None`: “nada”.

# Adelanto: diccionarios

## Datos con nombre, como una fila de una tabla

```
estudiante = {"nombre": "Ana", "edad": 20}
print(estudiante["nombre"])   # Ana
```

| NOMBRE | EDAD |
|---|---|
| Ana | 20 |



En la lista buscamos por **posición**; en el diccionario, por **nombre**. La próxima clase trabajamos con tablas completas.

# 6

# ¡A practicar!

Listas, bucles y funciones juntos

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301

UIFCE
UNIDAD DE INFORMÁTICA

# Errores frecuentes de hoy

## IndexError

```
notas = [3.5, 4.2, 2.8, 4.9]
notas[4]
IndexError: list index
out of range
```

La posición no existe: con 4 elementos, la última es la 3.

```
notas[3]
```

## NameError

```
for n in notas:
    total = total + n
NameError: name 'total'
is not defined
```

Al acumulador le falta su valor inicial.

```
total = 0
for n in notas:
    total = total + n
```

## SyntaxError

```
for n in notas
    print(n)
SyntaxError:
expected ':'
```

Al for le faltan los :, igual que al if.

```
for n in notas:
    print(n)
```

# Ejercicios

## Avanza hasta donde llegues

### Básico — TODOS

1. Crea una lista con 5 ciudades de Colombia e imprime cada una con un bucle.
2. Dada una lista de precios, calcula el total de la compra.

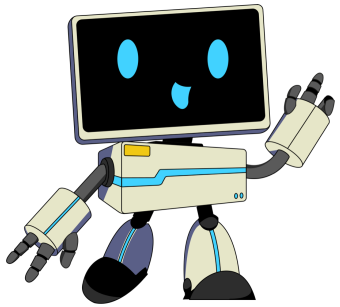
### Intermedio

3. Cuenta cuántas notas de una lista son mayores o iguales a 3.0.
4. Escribe una función `aplicar_iva(precio)` que devuelva el precio con IVA del 19 % y úsala sobre una lista de precios.

### Reto

5. Encuentra la nota más alta de una lista **sin** usar `max()`.
6. Escribe una función que reciba una frase y devuelva cuántas palabras tiene. Pista: `frase.split()`.

**¿Atascado? Haz la tabla de vueltas a mano.**



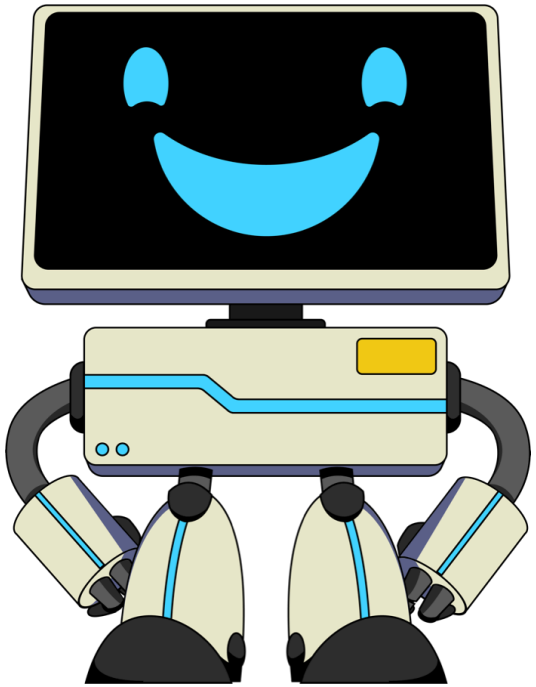
**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301



UIFCE
UNIDAD DE INFORMÁTICA

# Tres ideas clave de hoy

1. Las **listas** guardan varios datos en orden; en Python la primera posición es **0**.
2. El **for** repite una acción para cada elemento.
3. Las **funciones** empaquetan una solución para reutilizarla.



**Tarea corta**

Ejercicios 1 a 4.

**Nivelación**

Notebook resuelto sobre bucles y acumuladores.

**Próxima clase**

Usaremos un archivo CSV que compartiremos por enlace.

**Correo:** uniic_bog@unal.edu.co
**Teléfono:** 3165000 **ext** 12301



UIFCE
UNIDAD DE INFORMÁTICA

UIFCE
UNIDAD DE INFORMÁTICA

# ¡Gracias!

Próxima sesión: **Primer contacto con datos reales: pandas**.
¿Y si la lista tuviera miles de filas?

***Correo:*** uniic_bog@unal.edu.co
***Tel:*** 3165000 ***Ext:*** 12301

